Among Us klon (C++ / Qt): dijagram klasa po slojevima

UI → Klijent ⇄ TCP + UDP ⇄ Server → Logika igre. TCP: događaji; UDP: kretanje. Autoritativni server odlučuje.

UI SLOJ (kod svakog igrača)

MainWindow
QStackedWidget; prebacuje ekrane na phaseChanged; pri Host-u pravi Server + GameController

StartScreen
ime, Host / Join, IP adresa; emituje hostClicked / joinClicked

LobbyScreen
lista igrača, dugme Start (samo host)

GameScreen
QGraphicsView + scena: mapa, igrači, tela; HUD (Use / Kill / Report)
+ crveno dugme na mapi (hitni sastanak)

0..*

otvara

PlayerItem
QGraphicsItem

TaskDialog
minigra taska

otvara

EmergencyDialog
mali prozor sa velikim crvenim dugmetom (Emergency button); otvara ga GameScreen

VotingScreen
lista igrača, glasaj / Skip, tajmer

ChatWidget
QLineEdit + QTextEdit (chat tokom sastanka)

GameOverScreen
pobednik i uloge svih igrača

KLIJENT (kod svakog igrača)

lokalna kopija GameState kod klijenta (ista klasa, samo za prikaz)

ClientController
Prevodi akcije UI-ja u Message (requestKill, sendVote, sendChat...) i Message u signale (stateSnapshotReceived, phaseChanged, chatReceived, voteResultReceived).
+ callEmergencyMeeting()

send(Message)

signal messageReceived

Client
QTcpSocket + QUdpSocket; send(Message) bira protokol. TCP za događaje (i EmergencyMeeting); UDP za MoveInput i PositionSnapshot.

Veza UI ↔ kontroler:
UI zove metode kontrolera, kontroler javlja UI-ju signale (stateSnapshotReceived...).

koristi

SERVER (samo kod hosta)

GameController
SUDIJA. Prima poruke iz oba protokola. Validira kretanje i akcije; menja GameState. Šalje pozicije preko UDP-a (~50 ms), a stanje i događaje preko TCP-a.
+ handleEmergency(): validira i pokreće sastanak bez tela

broadcast / sendTo

signal messageReceived

Server
QTcpServer + QUdpSocket; playerId → PlayerNetworkSession; broadcast(), sendTo(id).

send(Message)

signal messageReceived

UDP JSON

TCP

ClientConnection
Jedan po igraču: QTcpSocket + playerId; sklapa TCP poruke, emituje messageReceived i disconnected.

0..*

PlayerNetworkSession
tcpConnection + udpAddress + udpPort + udpToken
Zapis u Server.m_players; jedan zapis po playerId.

koristi Message

pravi i vodi sesiju

LOGIKA IGRE (model, bez UI i mreže)

menja

GameState
igrači, mrtva tela, GamePhase, brojač taskova; checkWinCondition(); toJson / fromJson
+ emergencyButtonPos() (iz GameMap)

4..10

Player
id, ime, boja, pozicija, Role, alive, killCooldown
+ emergencyLeft (preostali hitni sastanci)

* (dodeljeni taskovi)

Task
id, name, location; createWidget(); isCompleted(); signal completed

WireTask
spajanje žica (i ostali taskovi kao podklase)

DeadBody
pozicija i id žrtve

GameMap
jpg (prikaz) + crno-bela maska; isWalkable(), taskStations(), spawnPoints()
+ emergencyButtonPos(): centar mape

VotingSession
castVote(), skipVote(), tally(); tajmer; lista ChatMessage
bodyId = -1: hitni sastanak (dugme)

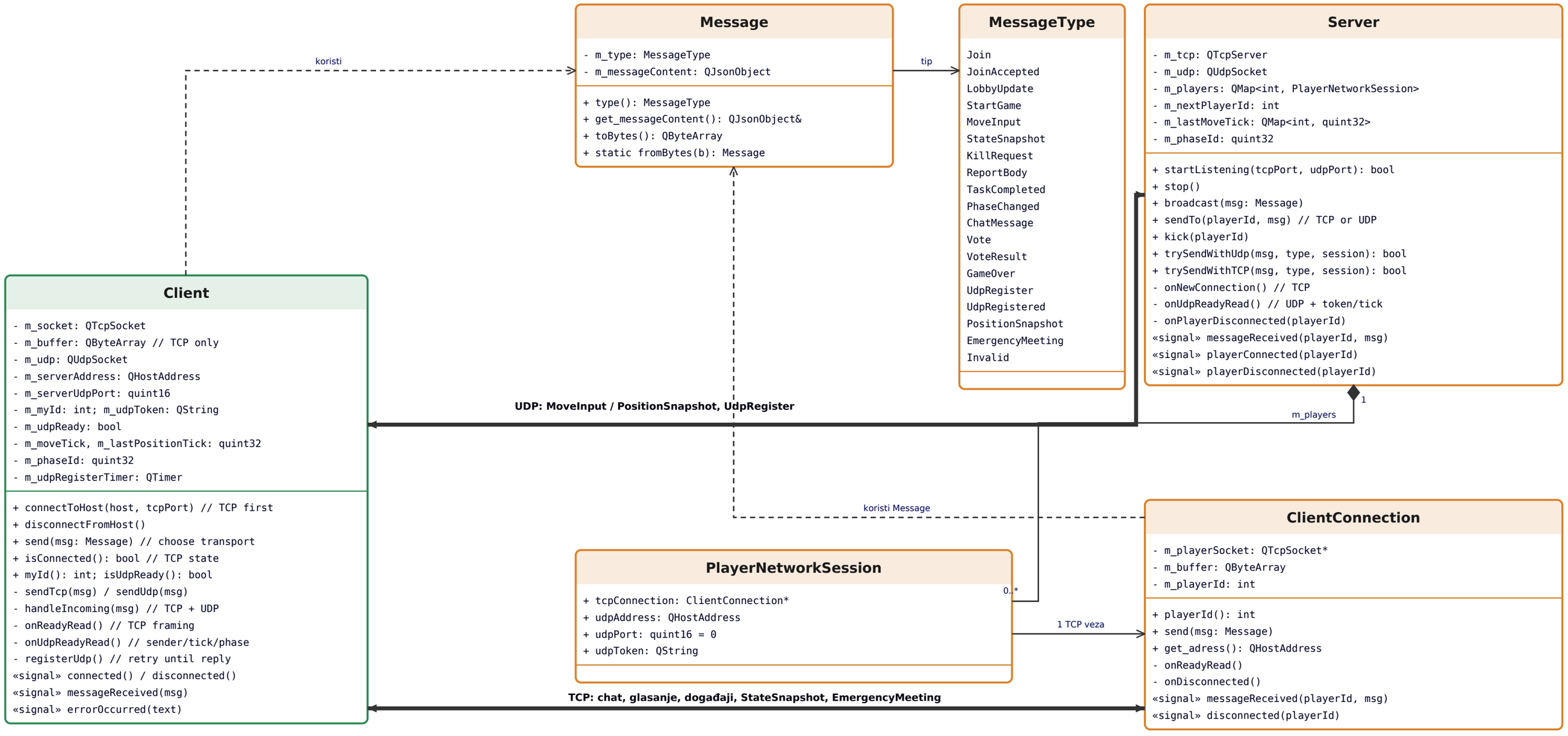
ChatMessage
senderId, tekst, vreme

Message
MessageType + JSON sadržaj; toBytes / fromBytes. TCP: [4 B dužine][JSON]. UDP: jedan JSON datagram, bez TCP zaglavlja; playerId + token + tick + phaseId.
Novi tip: MessageType::EmergencyMeeting (TCP).

Legenda:
poziv metode · signal / događaj · TCP/UDP · kompozicija (poseduje) · agregacija (koristi / ima) · nasleđivanje · zavisnost

Mrežni sloj: klase sa atributima i metodama

Jedan mrežni sloj: TCP za pouzdane događaje; UDP za kretanje. Server čuva playerId → PlayerNetworkSession.



TCP i životni vek igrača

JoinAccepted preko TCP-a nosi playerId, udpToken i serverUdpPort. TCP: [4 B dužine][JSON]; ClientConnection sklapa cele poruke. Prekid TCP-a: ukloni zapis iz m_players, deleteLater() konekcije, pa playerDisconnected(id). Socket je dete konekcije.

UDP registracija i zaštita od starih podataka

Client bind-uje svoj UDP port i ponavlja UdpRegister(playerId, token). Server proveri token, upiše adresu/port datagrama i vrati UdpRegistered. udpPort = 0: još nije spreman. Zatim proverava endpoint, token, phaseId i noviji tick; gubitak paketa nadoknađuje sledeći input/pozicija.

Isti interfejs prema igri

Client.send / Server.sendTo biraju transport po tipu poruke. UDP: samo MoveInput, PositionSnapshot i UDP registracija. StateSnapshot i svi događaji idu TCP-om. UDP menja samo pozicije, nikada role, alive ili taskove.
EmergencyMeeting ide TCP-om: jedan događaj koji ne sme da se izgubi.

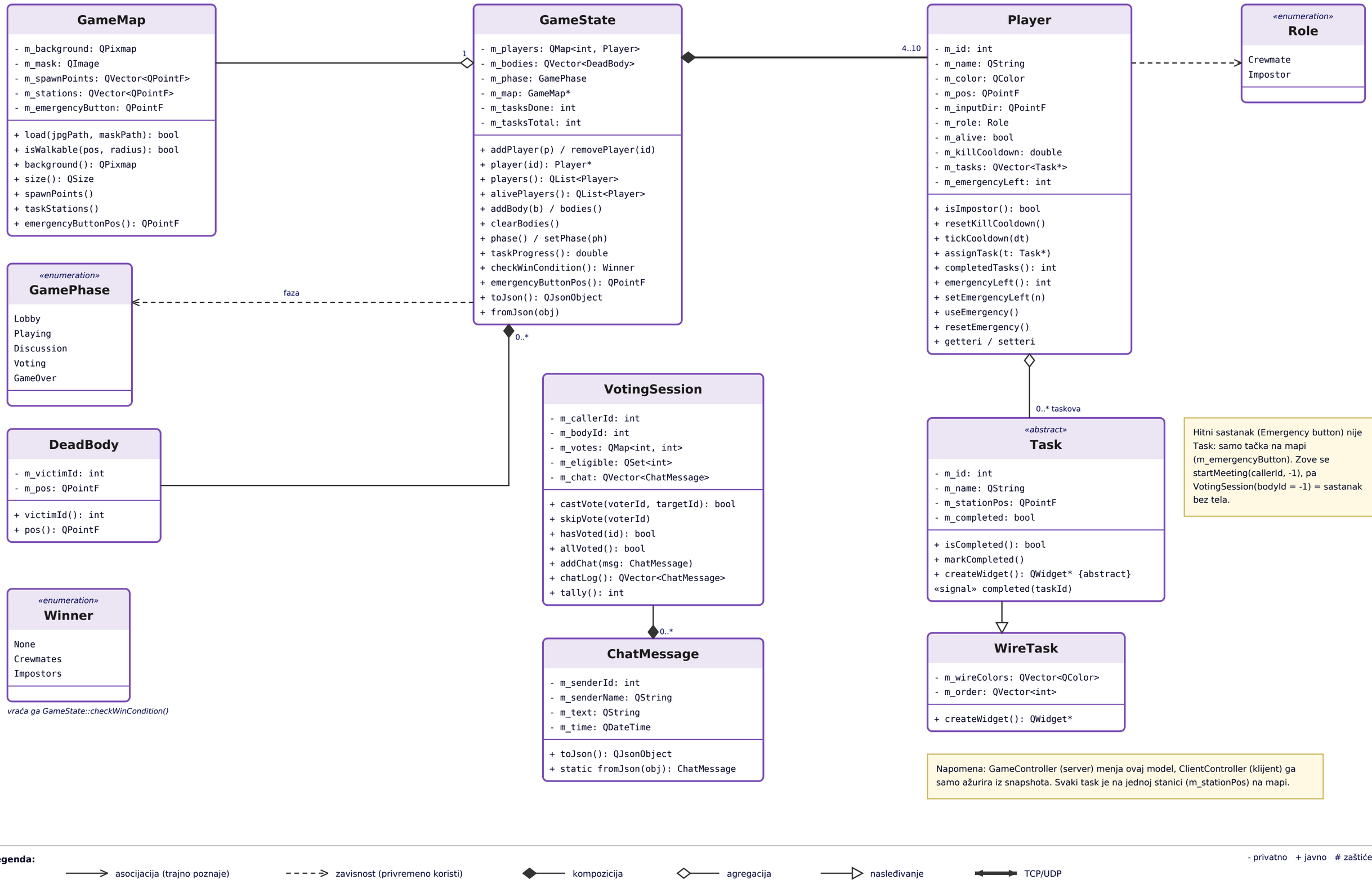
Legenda:

asocijacija (trajno poznaje) · zavisnost (privremeno koristi) · kompozicija · agregacija · naslеđivanje · TCP/UDP

- privatno + javno # zaštićeno

Model (logika igre): klase sa atributima i metodama

Ove klase ne znaju ni za UI ni za mrežu. Server ih menja kao istinu, klijent drži kopiju za prikaz.



Kontroleri: ClientController i GameController

Nemaju direktnu vezu. Razmenjuju samo Message preko mreže; svaki ima svoj lanac klasa ispod sebe.

Kill: UI → ClientController.requestKill() → Client.send() → TCP → ClientConnection → Server → GameController.onMessage() → handleKill(): validacija → GameState → sendSnapshot() preko TCP-a. Kretanje: sendMove → UDP → Server → handleMove / tick(). sendPositions() → UDP PositionSnapshot → lokalne pozicije → UI. StateSnapshot / PhaseChanged preko TCP-a usklađuju stanje i phaseId.

Oba transporta aktiviraju isti onMessage(playerId, msg). ClientController.onMessage obrađuje i PositionSnapshot: samo pozicije. Javni UI interfejs i model ostaju isti; phaseId se menja sa fazom igre.

Dugmad Use / Kill: server odlučuje, klijent samo prikazuje. sendSnapshot() za svakog igrača posebno (sendTo) računa nearTaskId (TASK_RANGE), canKill (KILL_RANGE, cooldown, uloga) i canEmergency, i šalje ih u njegovom StateSnapshot-u. ClientController ih samo prosledi UI-ju. Server ponovo proverava kad stigne zahtev (completeTask / requestKill / EmergencyMeeting).

Hitni sastanak: GameScreen (Use kod dugmeta) → EmergencyDialog → ClientController.callEmergencyMeeting() → TCP → GameController.handleEmergency(): canCallEmergency() (faza Playing, igrač živ, emergencyLeft > 0, cooldown istekao, razdaljina <= EMERGENCY_RANGE) → startMeeting(callerId, -1) → PhaseChanged(Discussion).

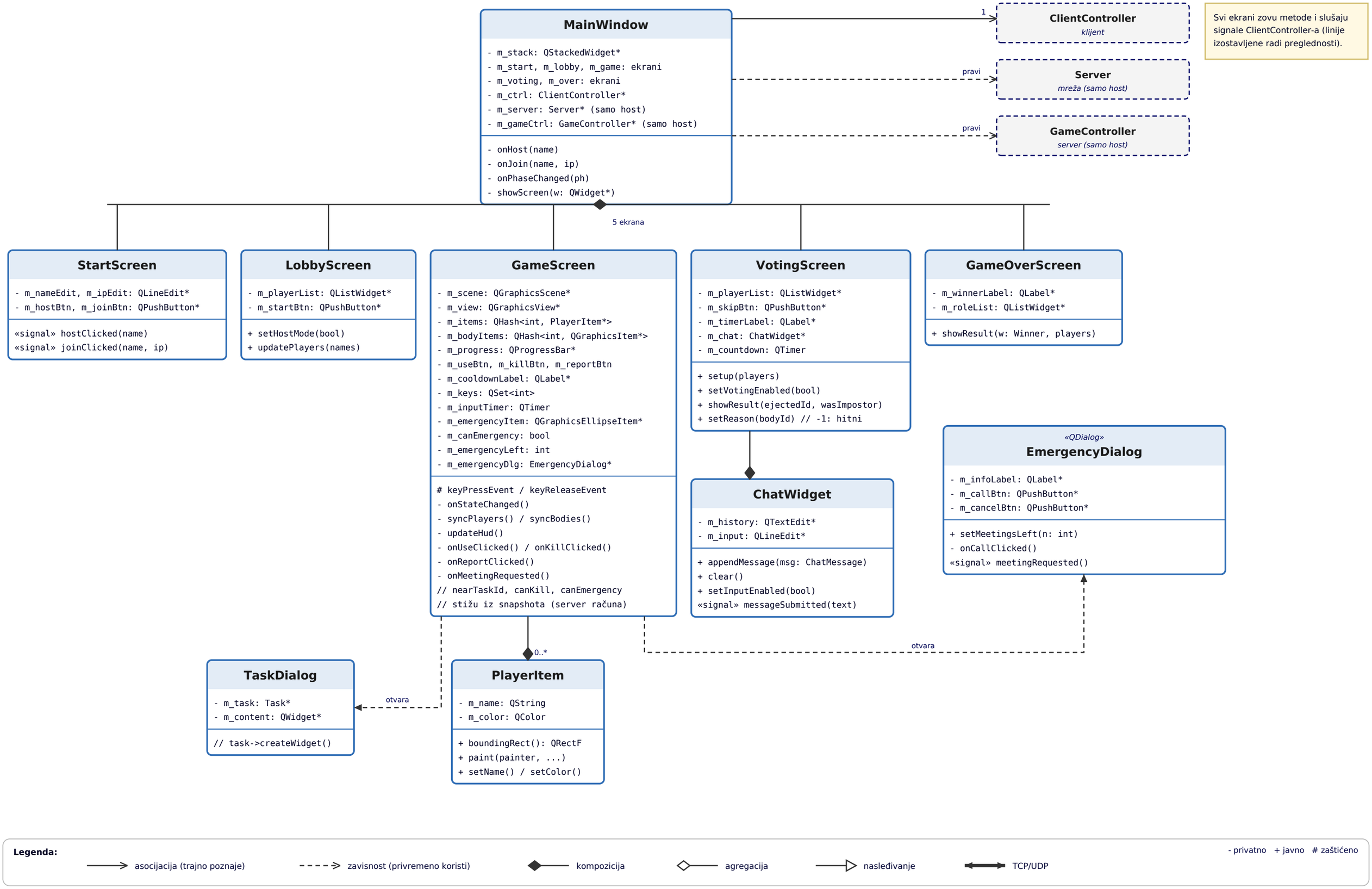
Legenda:

asocijacija (trajno poznaje) — zavisnost (privremeno koristi) — kompozicija — agregacija — nasleđivanje — TCP/UDP

- privatno + javno # zaštićeno

UI sloj: klase sa atributima i metodama

UI zove metode ClientController-a i sluša njegove signale; ne zna ništa o mreži.



Legenda:

asocijacija (trajno poznaje) — zavisnost (privremeno koristi) — kompozicija — agregacija — nasleđivanje — TCP/UDP

- privatno + javno # zaštićeno